UNIVERSITAS PAHLAWAN TUANKU TAMBUSAI
FAKULTAS TEKNIK — PROGRAM STUDI TEKNIK INFORMATIKA

# PANDUAN PRAKTIKUM PERTEMUAN 6

Pewarisan: Satu Induk, Tiga Jenis Alat

| | |
|---|---|
| **Mata Kuliah** | Pemrograman Berorientasi Objek · 4 sks · Semester III |
| **Sub-CPMK** | Sub-CPMK-6 — Menerapkan pewarisan dan penimpaan method khusus __str__, __repr__, __eq__, dan __hash__, serta menimbang komposisi sebagai alternatif pewarisan. |
| **Bobot terhadap nilai akhir** | 4% — praktikum 1% + Tugas 2 sebesar 3% |
| **Alokasi waktu** | Terbimbing 270 menit (kuliah 100 + praktikum 170) · Penugasan terstruktur 205 menit · Belajar mandiri 200 menit |
| **Profil gerbang** | p06 — minimal tiga kelas, ada pewarisan, dan ada penimpaan __str__ atau __eq__ |
| **Bekal dari Pertemuan 5** | Repositori bertanda versi p05 dengan Sewa, BarisSewa, dan diagram kelas. |
| **Tenggat hari ini** | Proyek Milestone 1 dikumpulkan sebelum praktikum dimulai, tanda versi m1. |
| **Luaran pertemuan** | Hierarki AlatTani dengan tiga turunan dan empat method khusus. Bertanda versi p06. |

## 1. PERSOALAN HARI INI

Sampai Pertemuan 5, traktor, pompa air, dan mesin perontok kita perlakukan sebagai satu kelas yang sama. Padahal ketiganya dihitung dengan cara yang berbeda di lapangan:

- Traktor wajib dioperasikan petugas UPJA, sehingga ada ongkos operator per hari.
- Pompa air memakai solar, dan pompa berdebit besar lebih boros daripada yang kecil.
- Mesin perontok punya biaya angkut yang besar, sehingga UPJA menetapkan minimum sewa dua hari.

Menambahkan tiga percabangan if di dalam biaya_sewa mungkin terpikir lebih dulu. Tetapi setiap kali UPJA menambah jenis alat baru, percabangan itu bertambah panjang, dan setiap penambahan berisiko merusak yang sudah jalan. Pewarisan menawarkan jalan lain.

## 2. TUJUAN DAN INDIKATOR

Setelah menyelesaikan pertemuan ini Anda mampu:

1. Merancang hierarki kelas yang memindahkan bagian bersama ke induk dan menyisakan yang khas di turunan.
2. Memanggil super() dengan benar, dan menjelaskan apa yang runtuh bila ia terlupa.
3. Menimpa method induk dengan tiga cara berbeda: menambah komponen, menghitung dari atribut sendiri, dan mengubah masukan sebelum meneruskan.
4. Menulis __str__ dan __repr__ dengan pembagian tugas yang tepat.
5. Menulis __eq__ bersama __hash__, dan menjelaskan mengapa keduanya tidak boleh dipisah.
6. Menimbang kapan pewarisan justru merugikan dan komposisi lebih tepat.

## 3. TIGA CARA MENIMPA METHOD INDUK

Menimpa bukan berarti menulis ulang dari nol. Tiga pola berikut semuanya memakai kembali pekerjaan induk, hanya berbeda pada bagian mana yang disesuaikan.

| Turunan | Pola Penimpaan | Wujudnya dalam Kode |
|---|---|---|
| Traktor | Menambah komponen biaya sesudah induk selesai | `dasar = super().biaya_sewa(hari); lalu tambahkan ongkos operator` |

| Turunan | Pola Penimpaan | Wujudnya dalam Kode |
|---|---|---|
| PompaAir | Menambah komponen yang dihitung dari atribut miliknya sendiri | `dasar = super().biaya_sewa(hari); lalu tambahkan solar sesuai debit` |
| Perontok | Mengubah masukan sebelum diteruskan ke induk | `return super().biaya_sewa(self.hari_ditagih(hari))` |

**Pola ketiga yang paling sering luput.** Perontok tidak menambah apa pun pada hasil perhitungan induk. Ia hanya memperbaiki angka masukannya sebelum diteruskan. Kalau disewa satu hari, yang ditagih tetap dua hari. Cara ini lebih bersih daripada menghitung ulang seluruh rumus di dalam turunan, dan ia otomatis ikut benar bila rumus induk suatu hari berubah.

## 4. EMPAT METHOD KHUSUS DAN PEMBAGIAN TUGASNYA

| Method | Untuk Siapa | Kapan Dipanggil |
|---|---|---|
| `__str__` | Untuk dibaca manusia. Rapi, tanpa tanda kurung teknis. | print(alat), f"{alat}", str(alat) |
| `__repr__` | Untuk dibaca pemrogram. Menyebut nama kelas dan nilai atributnya. | repr(alat), dan saat objek muncul di dalam daftar atau saat menelusuri galat |
| `__eq__` | Menentukan kapan dua objek dianggap sama. | alat_a == alat_b, dan pemeriksaan in pada daftar |
| `__hash__` | Menentukan letak objek di dalam set dan dict. | set(daftar), dict berkunci objek, pemeriksaan in pada set |

**__eq__ dan __hash__ selalu berpasangan.** Begitu sebuah kelas menulis ulang __eq__, Python otomatis menetapkan __hash__ menjadi None. Objeknya tetap bisa dibandingkan, tetapi tidak lagi bisa dimasukkan ke dalam set maupun dipakai sebagai kunci dict. Aturan sederhananya: bila dua objek dinyatakan sama oleh __eq__, keduanya wajib menghasilkan hash yang sama pula. Karena itu keduanya dihitung dari data yang sama — pada SIPALA, dari kode alat.

## 5. ALUR PERTEMUAN

| Waktu | Kegiatan | Bentuk |
|---|---|---|
| 0–20′ | Pengumpulan Milestone 1, pembukaan, dan kuis awal | Verifikasi berkas; LKM Bagian A |
| 20–65′ | Blok 1: hierarki, super(), dan tiga pola penimpaan | Kuliah dan live coding |
| 65–105′ | Blok 2: empat method khusus dan pasangan eq-hash | Kuliah dan demonstrasi |
| 105–115′ | Istirahat | — |
| 115–285′ | Blok 3: praktikum terbimbing, Langkah 1 sampai 9 | Praktikum; LKM Bagian B |
| 285–305′ | Blok 4: kapan pewarisan justru merugikan | Diskusi kelompok; LKM Bagian C |
| 305–320′ | Blok 5: penutup dan Tugas 2 | Refleksi; LKM Bagian D |

## 6. LANGKAH PRAKTIKUM

Kerjakan berurutan pada repositori yang sama. Tiga kelas baru ditambahkan hari ini, dan kelas AlatTani yang lama menjadi induknya.

**1** 10′

### Menyiapkan repositori

```
.venv\Scripts\activate        # Windows
source .venv/bin/activate     # Linux atau macOS
git pull
python periksa_pbo.py src --profil p05
python -m pytest
```

**2** 25′

### Membuat Traktor sebagai turunan AlatTani

Buat berkas src/model/traktor.py. Perhatikan baris pertama konstruktornya — sebelum mengerjakan apa pun, ia menyerahkan pekerjaan induk kepada induknya.

```
from typing import Final

from src.model.alat_tani import AlatTani


class Traktor(AlatTani):
    """Traktor roda dua. Wajib dioperasikan petugas UPJA."""

    ONGKOS_OPERATOR_HARIAN: Final[int] = 100_000

    def __init__(self, kode: str, nama: str, tarif_harian: int,
                 tenaga_kuda: int) -> None:
        super().__init__(kode, nama, tarif_harian)      # ← WAJIB, dan duluan
        if tenaga_kuda <= 0:
            raise ValueError(
                f"tenaga kuda harus lebih dari nol, diterima {tenaga_kuda}")
        self._tenaga_kuda = tenaga_kuda

    def biaya_sewa(self, jumlah_hari: int) -> int:
        """Biaya dasar ditambah ongkos operator untuk setiap hari sewa."""
        dasar = super().biaya_sewa(jumlah_hari)
        return dasar + Traktor.ONGKOS_OPERATOR_HARIAN * jumlah_hari
```

Dua hal yang perlu Anda sadari. Pertama, seluruh validasi tarif dari Pertemuan 3 tetap berlaku pada Traktor tanpa satu baris pun ditulis ulang — karena super().__init__() yang menjalankannya. Kedua, method biaya_sewa memanggil versi induknya lebih dulu, lalu menambahkan bagiannya sendiri. Menimpa bukan berarti membuang.

**3**
15′

### Kesalahan yang disengaja ①: super() yang terlupa

Buat berkas sementara lupa_super.py, lalu jalankan.

```
class Traktor(AlatTani):
    def __init__(self, kode: str, nama: str, tarif_harian: int,
                 tenaga_kuda: int) -> None:
        self._tenaga_kuda = tenaga_kuda        # super() TERLUPA
```

Keluaran:

```
Objek berhasil dibuat  : Traktor
Tenaga kuda terbaca    : 8
Isi objek              : ['_tenaga_kuda']

keterangan()    : RUNTUH -> 'Traktor' object has no attribute '_kode'
biaya_sewa(3)   : RUNTUH -> 'Traktor' object has no attribute '_tarif_harian'
```

**Yang runtuh bukan baris yang salah.** Objeknya terbentuk tanpa keluhan sedikit pun, dan atribut miliknya sendiri terbaca dengan benar. Galat baru muncul jauh kemudian, di dalam method warisan yang tidak pernah Anda sentuh. Perhatikan baris ketiga keluaran: vars() menunjukkan objek itu hanya punya satu atribut, padahal seharusnya empat. Inilah yang sesungguhnya dikerjakan super().__init__() — ia membangun bagian induk dari objek Anda. Catat pada LKM Bagian B.1.

Sekarang jalankan Ruff terhadap berkas itu:

```
python -m ruff check lupa_super.py
→ ARG002 Unused method argument: `kode`
→ ARG002 Unused method argument: `nama`
→ ARG002 Unused method argument: `tarif_harian`
```

Perhatikan bahwa Ruff tidak berkata “Anda lupa super()”. Ruff tidak memahami pewarisan. Yang ia lihat hanyalah tiga parameter yang diterima tetapi tidak pernah dipakai — itu gejalanya, bukan penyakitnya. Perkakas sering menolong dengan cara seperti ini: menunjuk ke arah yang benar tanpa tahu apa yang sesungguhnya salah.

4
30′

## Membuat PompaAir dan Perontok

Dua turunan berikut memakai pola penimpaan yang berbeda dari Traktor. Buat src/model/pompa_air.py terlebih dahulu.

```
class PompaAir(AlatTani):
    """Pompa air bermesin. Penyewa dikenakan biaya solar sesuai debitnya."""

    HARGA_SOLAR_PER_LITER: Final[int] = 7_000
    LITER_PER_SERATUS_DEBIT: Final[int] = 3

    def konsumsi_solar_harian(self) -> int:
        """Perkiraan pemakaian solar dalam liter untuk satu hari."""
        return (self._debit * PompaAir.LITER_PER_SERATUS_DEBIT) // 100

    def biaya_sewa(self, jumlah_hari: int) -> int:
        dasar = super().biaya_sewa(jumlah_hari)
        solar = self.konsumsi_solar_harian() * PompaAir.HARGA_SOLAR_PER_LITER
        return dasar + solar * jumlah_hari
```

Lalu src/model/perontok.py, yang memakai pola ketiga — mengubah masukan, bukan menambah hasil.

```
class Perontok(AlatTani):
    """Mesin perontok padi. Biaya angkutnya besar, jadi ada minimum sewa."""

    MINIMUM_HARI_SEWA: Final[int] = 2

    def hari_ditagih(self, jumlah_hari: int) -> int:
        """Hari yang benar-benar ditagih, memperhitungkan batas minimum."""
        return max(jumlah_hari, Perontok.MINIMUM_HARI_SEWA)

    def biaya_sewa(self, jumlah_hari: int) -> int:
        """Menagih sekurang-kurangnya dua hari, meski disewa lebih singkat."""
        return super().biaya_sewa(self.hari_ditagih(jumlah_hari))
```

Uji ketiganya berdampingan. Perhatikan bahwa perontok yang disewa satu hari tetap ditagih seperti dua hari:

```
[TR-01] Traktor Roda Dua Kubota — Rp150.000/hari · 8 HP · termasuk operator
    1 hari:   Rp250.000      3 hari:   Rp750.000
[PA-01] Pompa Air 3 Inci — Rp75.000/hari · 600 L/mnt · ±18 L solar/hari
    1 hari:   Rp201.000      3 hari:   Rp603.000
[PR-01] Mesin Perontok Padi — Rp200.000/hari · 900 kg/jam · minimum 2 hari
    1 hari:   Rp400.000      3 hari:   Rp600.000
```

**5**
15′

### Menulis __str__ dan __repr__

Keduanya sama-sama mengubah objek menjadi teks, tetapi untuk pembaca yang berbeda. Tambahkan pada kelas induk AlatTani, sehingga ketiga turunan mewarisinya.

```
    def __str__(self) -> str:
        """Untuk dibaca manusia: dipakai print() dan f-string."""
        return self.keterangan()

    def __repr__(self) -> str:
        """Untuk dibaca pemrogram: dipakai saat menampilkan isi koleksi."""
        return (f"{type(self).__name__}(kode={self._kode!r}, "
                f"nama={self._nama!r}, tarif_harian={self._tarif_harian!r})")
```

Dua hal yang perlu diperhatikan. Tanda !r di dalam f-string meminta bentuk repr, sehingga teks tampil bertanda kutip dan lebih mudah ditelusuri. Sementara type(self).__name__ membuat repr menyebut nama kelas yang sesungguhnya — objek Traktor menampilkan “Traktor(...)”, bukan “AlatTani(...)”, meskipun kodenya ditulis di induk.

```
print(pompa)         →  [PA-01] Pompa Air 3 Inci — Rp75.000/hari · 600 L/mnt
print(repr(pompa))   →  PompaAir(kode='PA-01', nama='Pompa Air 3 Inci', ...)
```

**6**
20′

### Menulis __eq__ bersama __hash__

Pada UPJA, kode alat bersifat unik. Dua objek dengan kode yang sama merujuk alat yang sama, apa pun isi atribut lainnya. Itulah dasar perbandingannya.

```
    def __eq__(self, lain: object) -> bool:
        """Dua alat dianggap sama bila kodenya sama, karena kode itu identitas."""
        if not isinstance(lain, AlatTani):
            return NotImplemented
        return self._kode == lain._kode

    def __hash__(self) -> int:
        """Wajib ditulis bersama __eq__ agar alat tetap dapat dimasukkan set."""
        return hash(self._kode)
```

Baris NotImplemented bukan sekadar formalitas. Ia memberi tahu Python bahwa perbandingan ini tidak dikenali, sehingga Python berbalik menanyakan hal yang sama kepada objek di sisi kanan. Bila keduanya menyerah, barulah hasilnya dianggap tidak sama. Tanpa baris itu, membandingkan alat dengan teks biasa akan melempar galat, padahal seharusnya cukup menjawab “tidak sama”.

**7**
15′

### Kesalahan yang disengaja ②: __eq__ tanpa __hash__

Buat berkas sementara eq_tanpa_hash.py berisi kelas yang hanya menulis __eq__, lalu jalankan.

```
Perbandingan berhasil : True
Nilai __hash__ kelas  : None
Dimasukkan ke set     : GAGAL -> unhashable type: 'AlatTani'
Dijadikan kunci dict  : GAGAL -> unhashable type: 'AlatTani'
Dicari dengan in      : GAGAL -> unhashable type: 'AlatTani'
```

**Perhatikan baris kedua.** Nilai __hash__ kelas itu benar-benar None — bukan hilang, melainkan sengaja dimatikan Python. Begitu sebuah kelas menulis ulang __eq__, Python menganggap aturan kesamaannya sudah berubah, sehingga aturan hash lama tidak lagi sah dan harus Anda tetapkan sendiri. Akibatnya objek Anda tidak dapat masuk set maupun menjadi kunci dict. Ini akan sangat terasa pada Pertemuan 11, ketika seluruh alat dikelola di dalam koleksi. Catat pada LKM Bagian B.2.

Jalankan Ruff terhadap kedua berkas kesalahan hari ini, lalu bandingkan cara keduanya tertangkap:

```
python -m ruff check lupa_super.py
→ ARG002 Unused method argument      (menunjuk GEJALA)

python -m ruff check eq_tanpa_hash.py
→ PLW1641 Object does not implement `__hash__` method   (menunjuk PENYAKIT)
```

**8**
25′

### Menguji hierarki

Minimal lima puluh dua fungsi uji seluruhnya. Yang baru pada pertemuan ini adalah pengujian terhadap perilaku warisan dan method khusus.

```
def test_validasi_induk_tetap_berlaku_pada_turunan() -> None:
    with pytest.raises(ValueError):
        Traktor("TR-01", "Traktor", -5_000, 8)      # tarif minus


def test_perontok_menagih_minimum_dua_hari() -> None:
    r = Perontok("PR-01", "Perontok", 200_000, 900)
    assert r.biaya_sewa(1) == r.biaya_sewa(2)


def test_alat_dapat_menjadi_kunci_dict() -> None:
    a = Traktor("TR-01", "Traktor", 150_000, 8)
    catatan = {a: "sedang disewa"}
    assert catatan[Traktor("TR-01", "Lain", 160_000, 9)] == "sedang disewa"


def test_dibandingkan_dengan_bukan_alat() -> None:
    assert Traktor("TR-01", "Traktor", 150_000, 8) != "TR-01"
```

Uji pertama adalah yang paling penting. Ia membuktikan bahwa validasi yang ditulis di Pertemuan 3 masih menjaga turunan yang baru dibuat hari ini, tanpa satu baris pun disalin. Itulah keuntungan pewarisan yang sesungguhnya. Uji ketiga sekaligus membuktikan bahwa __eq__ dan __hash__ Anda konsisten satu sama lain.

**9**
15′

### Menjalankan ketiga gerbang dan mengumpulkan

```
python periksa_pbo.py src --profil p06     →  LOLOS  (8 kelas)
python -m mypy                              →  Success
python -m pytest                            →  52 passed
python -m src.main

git add .
git commit -m "P06: hierarki AlatTani dan empat method khusus"
git tag p06
git push origin main --tags
```

Hapus lupa_super.py dan eq_tanpa_hash.py sebelum commit.

## 7. AKTIVITAS KELOMPOK: KAPAN PEWARISAN JUSTRU MERUGIKAN

Berkelompok, 20 menit. Pewarisan terlihat menyenangkan, dan justru di situ bahayanya. Tidak setiap kemiripan pantas dijadikan hierarki.

| Tiga tanda bahwa pewarisan dipaksakan |
|---|
| 1. Turunan mewarisi method yang tidak masuk akal baginya, sehingga terpaksa dibiarkan kosong atau dibuat melempar galat. |
| 2. Hubungannya bukan “adalah sejenis” melainkan “punya”. Traktor adalah sejenis alat tani — itu pewarisan. Traktor punya mesin — itu komposisi. |
| 3. Hierarkinya lebih dari tiga tingkat, sehingga untuk memahami satu method Anda harus membuka empat berkas. |

Yang dikerjakan kelompok pada LKM Bagian C:

1. Tinjau diagram kelas proyek kelompok Anda. Adakah dua kelas yang tampak mirip dan tergoda untuk dijadikan induk-turunan?
2. Terapkan uji “adalah sejenis” pada pasangan itu. Tuliskan kalimat lengkapnya, lalu nilai apakah masuk akal.
3. Bila ternyata bukan pewarisan, tentukan relasi apa yang lebih tepat dan gambarkan ulang bagian itu.
4. Temukan satu kemiripan pada proyek Anda yang sebaiknya diselesaikan dengan komposisi, bukan pewarisan, beserta alasannya.

*Butir keempat bernilai paling tinggi. Kelompok yang menjawab “semua kemiripan kami memang pewarisan” kehilangan nilai butir ini, karena hampir tidak pernah ada rancangan yang seluruhnya cocok dijadikan hierarki.*

## 8. TUGAS 2 (INDIVIDU) — DINILAI TERPISAH

| | |
|---|---|
| **Tugas** | Bangun hierarki kelas Pegawai untuk sebuah instansi di Kabupaten Kampar, dengan tiga jenis turunan yang cara menghitung penghasilannya berbeda. |
| **Ketentuan hierarki** | Satu kelas induk dan tiga turunan. Setiap turunan wajib memanggil super().__init__() dan menambahkan sekurang-kurangnya satu atribut khas. |
| **Tiga pola penimpaan** | Ketiga turunan wajib memakai pola penimpaan yang berbeda: satu menambah komponen, satu menghitung dari atribut sendiri, satu mengubah masukan sebelum meneruskan ke induk. |
| **Method khusus** | Induk wajib menulis __str__, __repr__, __eq__, dan __hash__. Perbandingan didasarkan pada satu atribut yang benar-benar bersifat identitas. |
| **Melengkapi contoh** | Tambahkan pula dari_baris_csv pada PompaAir dan Perontok di repositori SIPALA Anda, mengikuti pola yang sudah ada pada Traktor. |
| **Pengujian** | Minimal dua belas fungsi uji, dan minimal empat membuktikan penolakan. Wajib ada uji yang membuktikan objek dapat menjadi kunci dict. |
| **Gerbang** | Wajib lolos periksa_pbo.py --profil p06, mypy, dan pytest. Tanda versi p06-tugas2. |
| **Tenggat** | Pukul 23.59 pada hari sebelum Pertemuan 7. |
| **Belajar mandiri** | Rujukan [R1] Bab 3–4, [R2] Bab 12–14, [R3] Item 37–40. |

## 9. DAFTAR PERIKSA SEBELUM MENINGGALKAN LABORATORIUM

| | |
|---|---|
| ☐ | Milestone 1 sudah dikumpulkan dengan tanda versi m1 sebelum praktikum dimulai. |
| ☐ | Ketiga turunan memanggil super().__init__() sebagai baris pertama konstruktornya. |
| ☐ | Ketiga turunan memakai pola penimpaan yang berbeda satu sama lain. |
| ☐ | Pengamatan super() yang terlupa sudah dicatat pada LKM Bagian B.1. |
| ☐ | Pengamatan __eq__ tanpa __hash__ sudah dicatat pada LKM Bagian B.2. |
| ☐ | Perbandingan hasil Ruff terhadap kedua kesalahan sudah dicatat pada LKM Bagian B.3. |
| ☐ | Induk memiliki __str__, __repr__, __eq__, dan __hash__. |
| ☐ | Ada uji yang membuktikan objek dapat dimasukkan set dan menjadi kunci dict. |

| | |
|---|---|
| ☐ | Minimal 52 fungsi uji; ketiga gerbang hijau dengan profil p06. |
| ☐ | Kedua berkas kesalahan sudah dihapus; tanda versi p06 sudah terdorong. |